

DOCUMENTO 03

Documentação técnica do projeto

Uma visita guiada pelo código, pelas pastas e pelos conceitos de programação por trás do site e do sistema de reservas do Espaço Cruzeiro.

Para: Matheus (dono do projeto)

Objetivo: aprender enquanto entende o que foi feito

Versão: 1.0

Sumário

1. Como ler este documento
2. Conceitos básicos — o que é um site moderno
3. As tecnologias do projeto, uma por uma
4. A estrutura de pastas
5. O que cada arquivo faz (tour guiado)
6. Os fluxos principais do sistema
7. Conceitos de programação que você aprendeu aqui
8. Como o projeto evolui daqui pra frente
9. Glossário rápido

1. Como ler este documento

Este PDF foi escrito pra você que **não programa**, mas quer entender o que está dentro do projeto que desenvolvi pro Espaço Cruzeiro. A ideia é dupla: servir como referência do sistema e, de quebra, te ensinar os conceitos de programação web mais comuns, com exemplos reais tirados do seu próprio código.

Como ele está organizado

- Começamos do zero, explicando o que é um site moderno e por que ele é diferente de uma página estática dos anos 2000.
- Depois apresentamos uma por uma as tecnologias usadas e pra que cada uma serve.
- Em seguida fazemos um **tour guiado** pela estrutura de pastas e pelos arquivos, explicando o papel de cada um.
- No final, voltamos em conceitos de programação que apareceram no caminho, pra fechar com chave de ouro.

Dica: não precisa ler tudo de uma vez. Use o sumário. Se uma seção parecer muito densa, pule, continue, e volte depois. Programação é feita de camadas — cada releitura revela algo novo.

2. Conceitos básicos — o que é um site moderno

2.1 Frontend vs Backend

Todo sistema web tem dois lados. O **frontend** é tudo que o usuário vê e clica: botões, formulários, menus. Ele roda dentro do navegador (Chrome, Safari, etc). O **backend** é tudo que acontece nos bastidores: salvar uma reserva no banco de dados, mandar um e-mail de confirmação, processar um pagamento. Ele roda em um servidor, longe do usuário.

O frontend conversa com o backend por meio de **requisições** (pede coisas) e **respostas** (recebe o resultado). É como um garçom: você (frontend) pede, a cozinha (backend) prepara, o garçom traz de volta.

2.2 O que é um banco de dados

Pense numa planilha do Excel, mas muito mais poderosa. O banco de dados guarda **tabelas** com linhas e colunas. No seu projeto, existe uma tabela de reservas, uma de disponibilidade, uma de clientes, e por aí vai. Quando alguém preenche o formulário de reserva, o backend escreve uma nova linha nessa tabela. Quando o admin abre a lista de reservas, o backend **lê** essas linhas.

2.3 Server-side rendering (SSR)

Antigamente, sites eram *arquivos HTML* estáticos. Hoje, muitos sites são **gerados na hora** pelo servidor. É o caso do seu: quando alguém acessa `/buffet/aniversario`, o servidor pega os dados do banco, monta a página com as fotos e preços atualizados, e manda pronta pro navegador. Isso é **SSR**, e é o padrão do Next.js que usamos.

2.4 Por que um site precisa de tanta coisa?

Você verá nas próximas páginas que o projeto usa várias bibliotecas. Isso não é frescura — cada uma resolve um problema que, sem ela, você teria que resolver na mão. Um exemplo: se você não usasse o **Zod** pra validar formulários, precisaria escrever manualmente centenas de `if/else` pra checar se o e-mail tem `@`, se o telefone tem 11 dígitos, etc. Multiplica isso por todas as partes do sistema e você entende por que a stack (conjunto de tecnologias) é grande.

3. As tecnologias, uma por uma

Todas as peças que compõem o seu projeto. Leia como se fosse um catálogo — cada uma cumpre um papel bem específico.

3.1 Next.js 16

É o **framework principal** do projeto. Pense num framework como um kit de ferramentas pré-montado: em vez de começar do zero, você ganha estrutura de pastas, roteamento (qual URL abre qual tela), renderização no servidor, otimização de imagens, e mais. O Next.js é construído em cima do React e é mantido pela Vercel.

A versão 16 é recente e introduziu várias mudanças em relação às anteriores. Por isso o arquivo `AGENTS.md` na raiz do projeto avisa: 'esta NÃO é a Next.js que você conhece'. Coisas como o arquivo `proxy.ts` (no lugar do antigo `middleware.ts`) e `params/searchParams` assíncronos são novidades.

3.2 React

É a biblioteca que desenha a tela. Em vez de manipular HTML na marra, você escreve **componentes** — pedacinhos reutilizáveis de interface, como um botão, um card de produto, um menu. O Next.js usa React por baixo dos panos.

3.3 TypeScript

Uma variação do JavaScript que obriga você a declarar o **tipo** de cada coisa: isto é um número, isto é um texto, isto é uma lista de reservas. Parece chato, mas é o que impede bugs bobos. Se você tentar somar um número com um texto, o TypeScript grita antes do código ir ao ar.

3.4 Tailwind CSS v4

É a biblioteca de estilos (cores, espaçamentos, fontes). Em vez de escrever CSS separado, você aplica **classes utilitárias** direto no HTML. Por exemplo: `bg-primary text-white px-4 py-2` vira um botão verde com texto branco e um pouco de padding.

3.5 Supabase

Um serviço pronto que entrega um **banco de dados** (Postgres), **autenticação** (cadastro e login de usuários) e **armazenamento de arquivos** (fotos, PDFs). É hospedado na nuvem — você não precisa gerenciar servidor de banco. No seu projeto, ele guarda reservas, clientes, datas bloqueadas, e autentica o administrador.

3.6 Drizzle ORM

Um **ORM** (Object-Relational Mapper) é um intérprete entre seu código TypeScript e o banco de dados. Em vez de escrever SQL na mão (*SELECT * FROM bookings WHERE ...*), você escreve em TypeScript (*db.select().from(bookings).where(...)*). O Drizzle traduz pra SQL no momento de rodar. Vantagem: o TypeScript confere se você está pedindo colunas que existem.

3.7 Mercado Pago

A plataforma de pagamento. Quando um cliente faz uma reserva, nosso backend cria uma **preferência de pagamento** no Mercado Pago e redireciona o cliente pra lá. Depois que ele paga, o Mercado Pago avisa nosso sistema por um **webhook** (uma 'chamada telefônica' automática entre sistemas).

3.8 Resend

Serviço de envio de e-mails transacionais. É ele que manda a confirmação de reserva, o aviso de pagamento, os lembretes. Usamos porque enviar e-mail pela sua conta pessoal cairia em spam e teria limite baixo.

3.9 Zod

Uma biblioteca de **validação**. Você descreve o formato esperado ("e-mail válido, telefone com 11 dígitos, data entre hoje e daqui a 1 ano") e o Zod confere se os dados recebidos batem com essa descrição. Se não baterem, ele rejeita e explica qual campo está errado.

3.10 date-fns

Biblioteca pra manipular datas sem dor de cabeça. Soma dias, formata ("15 de março de 2026"), compara intervalos. Evita que você precise decorar as manhas do JavaScript padrão, que tem uma API de datas notoriamente estranha.

4. A estrutura de pastas

Um projeto Next.js tem algumas pastas com nomes e papéis convencionados. Aqui vai o mapa geral do Espaço Cruzeiro.

```
espaco-cruzeiro/  
■■■■ app/ ← todas as páginas e rotas do site  
■ ■■■■ (site)/ ← agrupamento das páginas públicas  
■ ■■■■ admin/ ← painel administrativo  
■ ■■■■ api/ ← endpoints (webhooks, etc)  
■ ■■■■ layout.tsx ← layout raiz (html, head, body)  
■ ■■■■ globals.css ← estilos globais (Tailwind)  
■■■■ components/ ← componentes reutilizáveis  
■■■■ lib/ ← código compartilhado (DB, auth, etc)  
■ ■■■■ db/ ← Drizzle + schema  
■ ■■■■ supabase/ ← clients Supabase  
■ ■■■■ mercadopago/ ← integração com Mercado Pago  
■ ■■■■ email/ ← envio de e-mails via Resend  
■■■■ public/ ← imagens e assets estáticos  
■■■■ drizzle/ ← migrations do banco  
■■■■ pdfs/ ← (esta pasta) geradores de PDF  
■■■■ proxy.ts ← roda antes de cada requisição  
■■■■ .env.local ← suas chaves e senhas (NÃO commitar!)  
■■■■ package.json ← dependências do projeto  
■■■■ tsconfig.json ← configuração do TypeScript  
■■■■ next.config.ts ← configuração do Next.js
```

Por que tantas pastas?

Separar por responsabilidade é uma regra de ouro da programação. Tudo relacionado a páginas vai em *app/*. Tudo relacionado a código utilitário (conexão com banco, envio de e-mail, autenticação) vai em *lib/*. Componentes visuais reutilizáveis vão em *components/*. Quando você bate o olho no arquivo, já sabe a categoria só pelo caminho.

5. O que cada arquivo faz (tour guiado)

Agora vamos entrar nos arquivos mais importantes. Não vou listar os 200+ arquivos do projeto — só os que importam pra entender o todo. Se quiser, tenha o projeto aberto ao lado enquanto lê.

5.1 A pasta `app/` — as páginas e rotas

No Next.js 16, **cada pasta dentro de `app/` vira uma URL**. A pasta `app/buffet/aniversario/` vira a URL `seusite.com/buffet/aniversario`. Dentro de cada pasta, arquivos com nomes especiais têm funções específicas:

page.tsx	É a página em si. O que o usuário vê. Cada pasta precisa de um pra virar uma URL na web.
layout.tsx	É um 'molde' que envolve todas as páginas dentro da mesma pasta. Bom pra colocar coisas comuns pra todas as páginas.
loading.tsx	Aparece enquanto a página está carregando dados do servidor.
error.tsx	Aparece se acontecer um erro na página.
route.ts	Um endpoint de API (não é uma página visível). Usado em <code><api/*></code> pra receber requisições e retornar dados.

Exemplo real do projeto: `app/admin/reservas/page.tsx`

Essa é a tela de listagem de reservas no painel admin. Ela busca no banco todas as reservas, separa por status, renderiza uma tabela, e mostra filtros. O código inteiro tem menos de 180 linhas, graças às abstrações do Drizzle e do React.

```
export default async function BookingsListPage({ searchParams }) {
  const { status, q } = await searchParams;
  const db = getDb();
  const rows = await db.select({ ... }).from(bookings);
  return ( ...tabela em JSX... );
}
```

Perceba o **async**. Ele diz: 'essa função vai esperar por coisas demoradas (o banco de dados)'. E o **await** marca onde ela espera. Esse é o jeito moderno do JavaScript lidar com operações assíncronas — assunto da seção 7.

5.2 A pasta `app/admin/` — o painel

Tudo aqui só abre pra quem está logado como administrador. Existem páginas pra: dashboard com resumo, lista e detalhe de reservas, calendário visual, clientes, avaliações, cupons, bloqueios de data, e configurações. Cada uma mora na sua subpasta e tem um `page.tsx`.

5.3 A pasta `app/api/` — os endpoints

São as 'portas de entrada' do backend que não são páginas. Por exemplo: `app/api/webhooks/mercadopago/route.ts` é o endereço que o Mercado Pago chama quando um pagamento muda de status. O arquivo exporta funções chamadas *POST*, *GET* etc, cada uma respondendo a um método HTTP.

5.4 A pasta `lib/` — o código utilitário

O coração técnico. Nada aqui aparece direto pro usuário, mas tudo que está nas páginas depende dessas funções.

<code>lib/db/index.ts</code>	Cria a conexão com o banco (Drizzle + postgres-js).
<code>lib/db/schema.ts</code>	Define as tabelas em TypeScript: <code><i>bookings</i></code> , <code><i>customers</i></code> , <code><i>blockedDates</i></code>
<code>lib/supabase/server.ts</code>	Cria o cliente Supabase do lado do servidor. Também tem a função <code><i>isAdmin()</i></code> que
<code>lib/supabase/client.ts</code>	Cliente Supabase do lado do navegador (pra login, logout).
<code>lib/mercadopago/*</code>	Cria preferências de pagamento e lê notificações de webhook.
<code>lib/email/*</code>	Templates e envio de e-mails via Resend.
<code>lib/utils.ts</code>	Funções genéricas: formatar moeda (R\$), datas, etc.
<code>lib/zod/*</code>	Schemas de validação: o formato que cada formulário deve ter.

5.5 A pasta `components/`

Componentes reutilizáveis que aparecem em várias páginas: botões, cards, formulários, modal de confirmação. A ideia é: escreveu uma vez, usa cinquenta. Se um dia você mudar o estilo do botão, muda num lugar só.

5.6 O arquivo `proxy.ts`

Roda **antes** de cada requisição chegar na página. É onde verificamos se o usuário tem permissão pra acessar o `/admin/*`, por exemplo. No Next.js 16 esse arquivo chama `proxy.ts` (antes se chamava `middleware.ts`, e em muito material na internet você ainda verá o nome antigo — cuidado com desatualização).

5.7 O arquivo `.env.local`

É um arquivo de **variáveis de ambiente**. Guarda as chaves sensíveis: senha do banco, token do Mercado Pago, chave da Supabase. Ele **nunca** deve ir pro Git (por isso existe no `.gitignore`). Cada ambiente (seu PC, produção) tem o seu.

Regra de ouro: se um valor é um segredo, ele mora em `.env.local`. Se alguém pegar esse arquivo, tem acesso ao seu banco e às suas integrações. Trate com o mesmo cuidado que senha do banco.

5.8 O package.json

A carteira de identidade do projeto: nome, versão, e a lista de **dependências** (as bibliotecas que o projeto usa). Quando alguém roda *npm install*, o npm lê esse arquivo e baixa tudo que falta na pasta *node_modules/*.

6. Os fluxos principais do sistema

Até aqui vimos peças soltas. Agora juntamos. Um **fluxo** é a sequência de eventos que acontece quando alguém clica num botão ou acessa uma página.

6.1 Fluxo de reserva (o mais importante)

- 1. Cliente acessa o site e clica num tipo de evento (aniversário, casamento). O Next.js serve a página correspondente.
- 2. Cliente escolhe uma data no calendário. O calendário consulta `/api/availability` pra saber quais datas estão livres. Essa rota lê o banco (reservas + bloqueios) e responde com a lista de datas disponíveis.
- 3. Cliente preenche o formulário: nome, telefone, e-mail, convidados, extras (decoração, bebidas). O **Zod** valida no cliente e no servidor.
- 4. Ao enviar, o servidor cria a linha na tabela *bookings* com status *pending_payment* e gera um **código** único (ex: EC-2026-0042).
- 5. O servidor pede ao Mercado Pago uma 'preferência de pagamento' e redireciona o cliente pro checkout deles.
- 6. Cliente paga. O Mercado Pago chama nosso webhook em `/api/webhooks/mercadopago`. O webhook confere que o pagamento é real, atualiza o status da reserva pra *confirmed* e dispara um e-mail de confirmação via Resend.
- 7. Cliente cai numa página de agradecimento. Dali a minutos, recebe o e-mail. Fim do fluxo.

6.2 Fluxo de login do admin

- 1. Admin acessa `/admin/login` e digita e-mail e senha.
- 2. O Supabase confere as credenciais e, se baterem, devolve um **cookie** de sessão pro navegador.
- 3. Toda vez que o admin tenta abrir uma página `/admin/*`, o *proxy.ts* intercepta, confere o cookie, e verifica se o usuário tem a *role* "admin" no *app_metadata*.
- 4. Se sim, deixa passar. Se não, manda de volta pro login.

6.3 Fluxo de webhook (os pagamentos)

Webhooks são chamadas que serviços externos fazem pro nosso servidor. O Mercado Pago chama `/api/webhooks/mercadopago` várias vezes pra um mesmo pagamento — às vezes o mesmo evento chega duas ou três vezes. Por isso implementamos **idempotência**: o webhook registra cada evento recebido numa tabela e, se o mesmo ID chegar de novo, ignora. Assim a mesma reserva nunca é confirmada em duplicata.

Idempotência = 'rodar de novo dá o mesmo resultado'. É um princípio essencial sempre que dois sistemas se falam pela internet, porque a rede falha e mensagens se repetem.

7. Conceitos de programação que apareceram aqui

Agora a parte mais instrutiva. Vou destrinchar conceitos que ficaram implícitos nos arquivos que vimos. Se você tirar uma coisa só dessa seção, já valeu o PDF.

7.1 Síncrono vs assíncrono

Algumas operações são **rápidas**: somar dois números, inverter um texto. O código espera o resultado e segue. Isso é **síncrono**. Outras são **demoradas** em escala de computador: buscar algo no banco (milissegundos) ou no Mercado Pago (segundos). Se o programa ficasse travado esperando, ninguém mais conseguiria usar o site. Por isso essas são **assíncronas**: o código dispara a operação, vai fazer outras coisas, e quando o resultado chega, continua.

No TypeScript moderno usamos **async/await**. Uma função marcada como *async* pode usar *await* pra esperar um resultado assíncrono sem bloquear o resto.

```
async function carregarReservas() {  
  const rows = await db.select().from(bookings);  
  return rows;  
}
```

7.2 Server Components vs Client Components

No Next.js 16 (com React Server Components), os arquivos da pasta *app/* são, por padrão, **componentes de servidor**: rodam só no servidor, podem ler banco direto, **não** mandam nenhum JavaScript pro navegador. Quando você precisa de interação (cliques, campos controlados, animações), marca o arquivo com *"use client"* no topo, e ele vira **Client Component**.

Isso é diferente do que existia anos atrás e é uma das mudanças mais importantes do React recente. A ideia é: só mande JavaScript pro navegador quando realmente precisar. Páginas mais leves, site mais rápido.

7.3 Tipos (TypeScript)

Toda variável, parâmetro e retorno de função pode ser descrito com um **tipo**. Exemplo:

```
type Reserva = {  
  id: string;  
  nomeCliente: string;  
  dataEvento: Date;  
  convidados: number;  
  status: "pending_payment" | "confirmed" | "cancelled";  
};
```

O tipo *status* só aceita três valores — se você tentar colocar "pago", o compilador avisa. Esse rigor ajuda muito em projetos grandes.

7.4 Validação de dados com Zod

Todo dado que vem do mundo externo (formulário, URL, webhook) é suspeito. Nunca confie. O Zod é um 'porteiro':

```
const schema = z.object({
  email: z.string().email(),
  telefone: z.string().min(10),
  dataEvento: z.coerce.date(),
});

const resultado = schema.safeParse(dadosDoFormulario);
if (!resultado.success) { /* mostra erro */ }
```

7.5 ORM e o schema do banco

Em *lib/db/schema.ts* descrevemos cada tabela em TypeScript. Isso vira, ao mesmo tempo, a fonte da verdade pro banco e o tipo que o TypeScript usa pra conferir suas consultas. Quando você precisa mudar o banco, altera o schema e roda uma **migration** (um script que diz ao banco: "adicione essa coluna, remova aquela").

7.6 Autenticação e RLS

Autenticar é provar quem você é (login). Autorizar é definir o que você pode ver e fazer. O Supabase oferece **Row Level Security (RLS)**: regras direto no banco que dizem, linha por linha, quem pode ler ou gravar. Isso protege mesmo se alguém descobrir um bug no nosso código — o banco *também* bloqueia.

7.7 Variáveis de ambiente e 12-factor

Um dos 12 princípios (**12-factor app**) da boa arquitetura é: configuração fica fora do código. URL do banco, chaves de API, tudo em *.env*. Assim o mesmo código roda em dev, staging e produção só trocando variáveis.

7.8 Git e versionamento

Cada alteração fica registrada em **commits**. Você pode voltar no tempo, criar ramos paralelos (**branches**), mesclar mudanças. É a rede de segurança do programador. Mesmo projeto de uma pessoa só se beneficia: um erro hoje, você desfaz amanhã sem perder sono.

8. Como o projeto evolui daqui pra frente

8.1 Pedindo mudanças

Quando você quiser alterar algo (novo pacote, texto da capa, preço), me mande o pedido e eu altero. Alterações pequenas vão ao ar em minutos, basta refazer o **deploy** (um comando que manda a nova versão do código pro servidor).

8.2 Entendendo bugs

Um bug é um comportamento inesperado. Pra descobrir a causa, o programador usa **logs** (mensagens registradas nos bastidores) e ferramentas do navegador. Todo *try/catch* que você vê no código é uma rede de segurança: se der ruim, a gente captura o erro e registra, em vez de deixar o site cair.

8.3 Lendo documentação

Saber programar é, em boa parte, saber procurar na documentação. Cada biblioteca que citei nesse documento tem um site com exemplos. Se quiser começar a aprender, sugiro a ordem:

- HTML e CSS (o básico de qualquer página)
- JavaScript (a linguagem do navegador)
- TypeScript (pra depois de dominar o JS)
- React (componentes)
- Next.js (o framework que une tudo)

8.4 Recursos gratuitos

MDN Web Docs	A referência oficial de HTML/CSS/JS.
javascript.info	Curso gratuito, bem didático.
react.dev	Documentação oficial do React.
nextjs.org/docs	Guia do Next.js. Leia a versão 16!
youtube.com/@WebDevSimplified	Canal em inglês muito bom.
youtube.com/@CodandoComFrontend	Canal BR de qualidade.
roadmap.sh	Mapas visuais de aprendizado por área.

9. Glossário rápido

Termos que usamos nesse PDF (e que você vai ouvir no dia a dia de quem trabalha com web).

API	Interface que um sistema expõe pra outro conversar com ele.
Backend	A parte do sistema que roda no servidor.
Branch	Ramo paralelo de desenvolvimento no Git.
Cache	Cópia temporária de dados pra responder mais rápido.
Cliente (client)	Quem consome um serviço — normalmente o navegador.
Commit	Um ponto salvo no histórico do código.
Cookie	Pedacinho de dado que o servidor guarda no navegador do usuário.
CRUD	Create, Read, Update, Delete — as 4 operações básicas em dados.
Deploy	Ato de publicar uma nova versão do site no ar.
Endpoint	Um endereço (URL) específico que responde a requisições.
Framework	Kit pré-montado com estrutura e ferramentas (ex: Next.js).
Frontend	A parte do sistema que o usuário vê, no navegador.
Git	Sistema de controle de versão.
Hosting	Servidor que mantém o site no ar (ex: Vercel, Supabase).
HTTP/HTTPS	O protocolo que faz o navegador falar com o servidor.
JSON	Formato leve de dados usado pra trocar informação entre sistemas.
Middleware/Proxy	Código que roda antes de cada requisição.
Migration	Script que altera a estrutura do banco de dados.
Postgres	Banco de dados relacional open-source usado pelo Supabase.
React	Biblioteca pra construir interfaces em componentes.
Role	Papel de um usuário (ex: admin, cliente).
Rota	Uma URL atendida pelo sistema.
Servidor (server)	Máquina que hospeda e executa o backend.
SQL	Linguagem usada pra conversar com bancos relacionais.
Stack	O conjunto de tecnologias usadas no projeto.
Token	Cadeia de caracteres que prova identidade ou permissão.

Webhook

Chamada automática que um sistema faz pra outro quando algo acontece.

Se chegou até aqui: parabéns. Em pouco tempo você passou por conceitos que muita gente da área leva meses pra ver pela primeira vez. Use esse PDF como referência, volte quando precisar, e bom aprendizado.